

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»

Факультет программной инженерии и компьютерной техники

Базы данных
Лабораторная работа №3
Вариант 505881

Выполнила

Розова Елена Алексеевна

Р3110

Проверил

Николаев Владимирович Вячеславович

Преподаватель факультета ПИиКТ

г. Санкт-Петербург 2025г

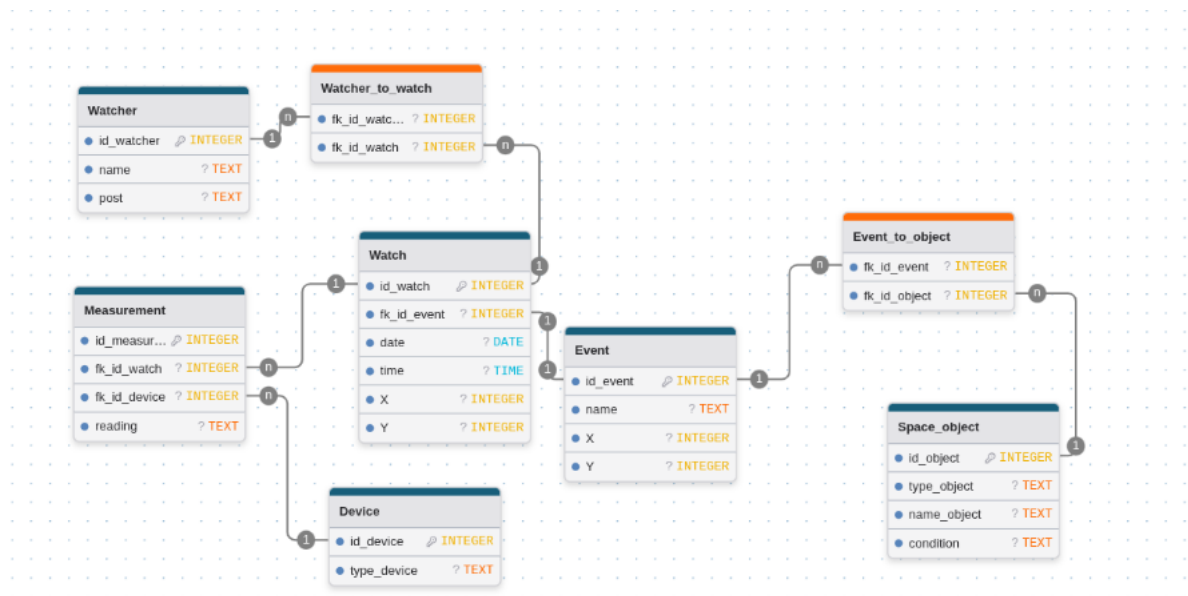
Текст задания

Для отношений, полученных при построении предметной области из лабораторной работы №1, выполните следующие действия:

- Опишите функциональные зависимости для отношений полученной схемы (минимальное множество);
- Приведите отношения в 3NF (как минимум). Постройте схему на основеNF (как минимум).
- Опишите изменения в функциональных зависимостях, произошедшие после преобразования в 3NF (как минимум). Постройте схему на основеNF;
- Преобразуйте отношения в BCNF. Докажите, что полученные отношения представлены в BCNF. Если ваша схема находится уже в BCNF, докажите это;
- Какие денормализации будут полезны для вашей схемы? Приведите подробное описание.

Придумайте триггер и связанную с ним функцию, относящиеся к вашей предметной области, согласуйте их с преподавателем и реализуйте на языке PL/pgSQL.

Даталогическая модели



Функциональные зависимости

Watcher: id_watcher -> name

id_watcher -> post

Watch: id_watch -> fk_id_event

id_watch -> date

id_watch -> time

id_watch -> x

id_watch -> y

Measurement: id -> fk_id_watch

id_watch -> fk_id_device

id_watch -> reading

Device: id_device -> type_device

Space_object: id_object -> type_object

id_watch -> name

id_watch -> object

id_watch -> condition

Event: id_event -> name

id_watch -> x

id_watch -> y

Соответствие нормам

1NF: Отношение находится в 1NF, если все его атрибуты содержат только атомарные значения. Моя модель удовлетворяет 1NF, так как все атрибуты атомарны, и нет повторяющихся групп.

2NF: Отношение находится во 2NF, если оно находится в 1NF и все его неключевые атрибуты полностью функционально зависят от первичного ключа. Моя модель удовлетворяет 2NF, так как все неключевые атрибуты полностью функционально зависят от первичных ключей.

3NF: Отношение находится в 3NF, если оно находится во 2NF и не содержит транзитивных зависимостей. Моя модель удовлетворяет 3NF, так как все неключевые атрибуты зависят только от первичных ключей, и не содержат транзитивных зависимостей.

BCNF: Отношение находится в BCNF, если для каждой функциональной зависимости $X \rightarrow Y$, X является суперключом. Моя модель удовлетворяет BCNF, так как для всех функциональных зависимостей X является суперключом.

Денормализация

Объединение таблиц может ускорить обработку запросов. Например, можно рассмотреть объединение таблиц Watch и Event, всё равно они связаны 1 к 1

Добавление избыточных атрибутов: В некоторых случаях добавление избыточных атрибутов может улучшить производительность запросов. Например, если часто запрашивается количество наблюдателей, то можно добавить атрибут number_watches в таблицу Watch. Это позволит избежать операций подсчета при каждом запросе/

Что делают триггеры:

- 1) При создании Watch создается отдельная таблица логирования, где считается расстояние между местом события и местом наблюдения.
- 2) Если происходит событие “взрыв”, то после этого идёт событие извержение света

Триггер

```
CREATE TABLE log_distance(  
log_id SERIAL PRIMARY KEY,  
id_watch INTEGER,  
id_event INTEGER,  
x_watch INTEGER,  
y_watch INTEGER,  
x_event INTEGER,  
y_event INTEGER,  
distance FLOAT  
);  
  
CREATE OR REPLACE FUNCTION distance()  
RETURNS TRIGGER AS $$  
DECLARE  
new_x_event INTEGER;  
new_y_event INTEGER;  
n_distance FLOAT;  
  
BEGIN  
if NEW.id_event IS NULL THEN  
RETURN NEW;  
END IF;  
  
SELECT x,y INTO new_x_event, new_y_event  
FROM Event  
WHERE id_event = NEW.id_event;  
  
n_distance := SQRT(POWER(NEW.x - new_x_event, 2) + POWER(NEW.y -  
new_y_event, 2));  
  
INSERT INTO log_distance(id_watch, id_event, x_watch, y_watch, x_event, y_event,  
distance)  
VALUES (NEW.id_watch, NEW.id_event, NEW.x, NEW.y, new_x_event, new_y_event,  
n_distance);  
  
RETURN NEW;  
END;  
$$ LANGUAGE PLPGSQL;
```

```
CREATE TRIGGER trg_log_dist
AFTER INSERT ON Watch
FOR EACH ROW
EXECUTE FUNCTION distance();

CREATE OR REPLACE FUNCTION explosion()
RETURNS TRIGGER AS $$
DECLARE
new_id INTEGER;
BEGIN
if NEW.name IS NULL THEN
RETURN NEW;
END IF;

if lower(NEW.name) LIKE '%взрыв%' THEN
SELECT COALESCE(MAX(id_event), 0) + 1 INTO new_id FROM Event;
INSERT INTO Event(id_event, name, x, y)
VALUES (new_id + 1, 'извержение света', NEW.x, NEW.y);
END IF;
RETURN NEW;
END;
$$ LANGUAGE PLPGSQL;

CREATE TRIGGER trg_log_explosion
AFTER INSERT ON Event
FOR EACH ROW
EXECUTE FUNCTION explosion();
```